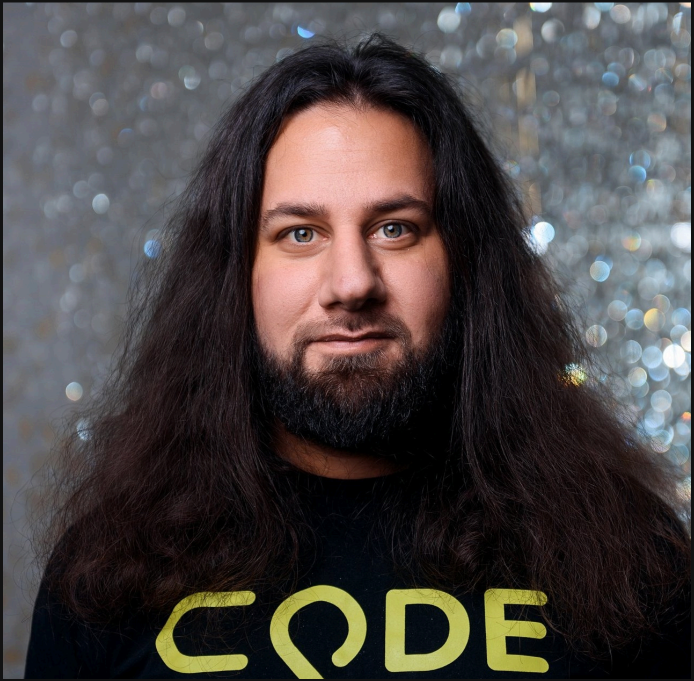
Advanced Kotlin Techniques for Spring Developers

Pasha Finkelshteyn

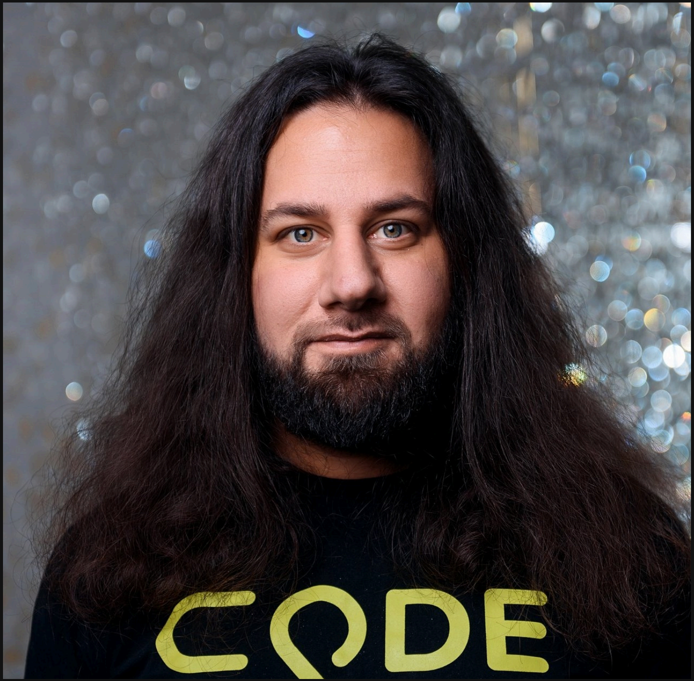
bellsoft

`whoami`



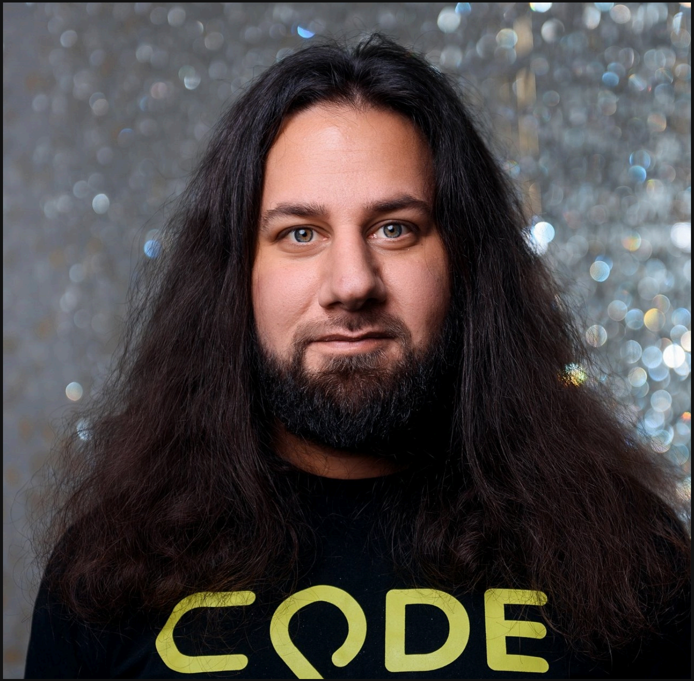
`whoami`

- Pasha Finkelshteyn



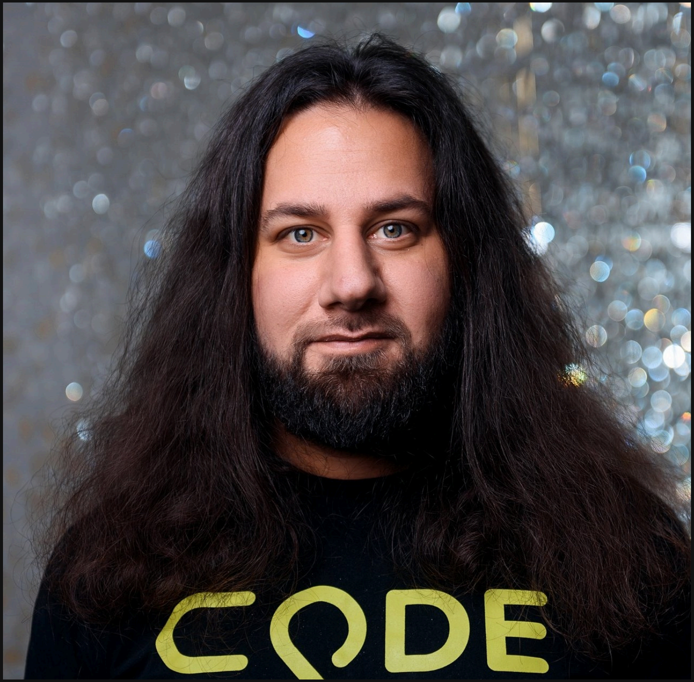
`whoami`

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft



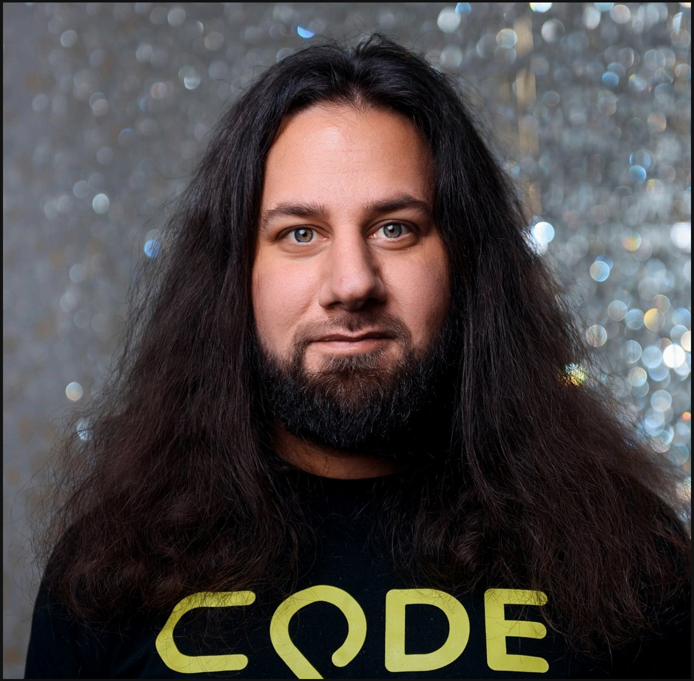
`whoami`

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and



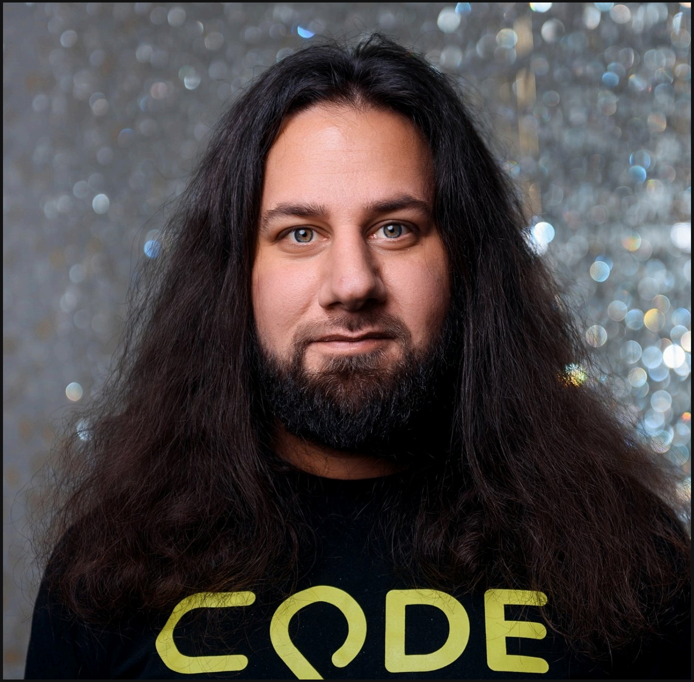
`whoami`

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and Kotlin
- And Spring



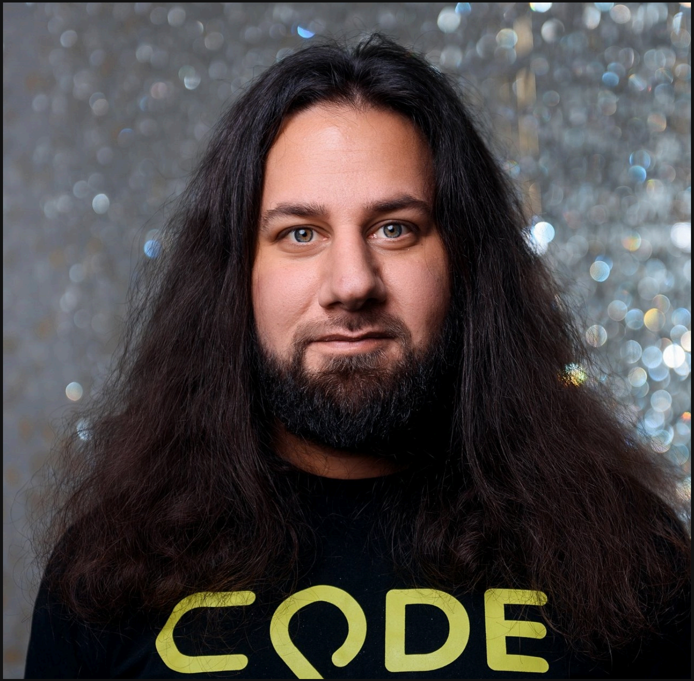
whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly and
- And
- asm0di0



`whoami`

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and Kotlin
- And Spring
- Twitter: asm0di0
- Mastodon: @asm0dey@fosstodon.org



BellSoft

- Vendor of Liberica JDK
- Contributor to the OpenJDK
- Author of ARM32 support in JDK

Liberica is the JDK officially recommended by

BellSoft

- Vendor of Liberica JDK
- Contributor to the OpenJDK
- Author of ARM32 support in JDK

Liberica is the JDK officially recommended by

We know our stuff!

bellsoft

bitly

I hope to showcase something you don't know yet!

My application

My application

- Simple nano-service

My application

- Simple nano-service
- MVC

My application

- Simple nano-service
- MVC
- Validation

My application

- Simple nano-service
- MVC
- Validation
- JPA

My application

- Simple nano-service
- MVC
- Validation
- JPA
- JDBC

My application

- Simple nano-service
- MVC
- Validation
- JPA
- JDBC
- Tests

Where do I start?

https://start.spring.io

Project

- ○ Gradle - Groovy
- ● Gradle - Kotlin
- ○ Maven

Language

- ○ Java
- ● Kotlin
- ○ Groovy

Spring Boot

- ○ 3.1.1 (SNAPSHOT)
- ● 3.1.0
- ○ 3.0.8 (SNAPSHOT)
- ○ 3.0.7
- ○ 2.7.13 (SNAPSHOT)
- ○ 2.7.12

Minimum dependencies

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Validation I/O
Bean Validation with Hibernate validator.

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Security SECURITY
Highly customizable authentication and access-control framework for Spring applications.

PostgreSQL Driver SQL
A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.

Testcontainers TESTING
Provide lightweight, throwaway instances of common databases, Selenium web browsers, or anything else that can run in a Docker container.

2 files are generated

- `build.gradle.kts`
- `SpringKotlinStartApplication.kt`

What happens

```
import org.jetbrains.kotlin.gradle.tasks.KotlinCompile

plugins {
  id("org.springframework.boot") version "3.3.0"
  id("io.spring.dependency-management") version "1.1.5"
  kotlin("jvm") version "1.9.24"
  kotlin("plugin.spring") version "1.9.24"
  kotlin("plugin.jpa") version "1.9.24"
}

group = "com.github.asm0dey"
version = "0.0.1-SNAPSHOT"

java {
  sourceCompatibility = JavaVersion.VERSION_21
}

repositories {
  mavenCentral()
```

Pasha

The main class

Main class

```kotlin
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
class SampleApplication

fun main(args: Array<String>) {
  runApplication<SampleApplication>(*args)
}
```

Main class

```kotlin
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
class SampleApplication

fun main(args: Array<String>) {
  runApplication<SampleApplication>(*args)
}
```

Main class

```kotlin
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
class SampleApplication

fun main(args: Array<String>) {
  runApplication<SampleApplication>(*args)
}
```

runApplication

```
inline fun <reified T : Any> runApplication(vararg args: String) =
    SpringApplication.run(T::class.java, *args)
```

runApplication

```kotlin
inline fun <reified T : Any> runApplication(vararg args: String) =
        SpringApplication.run(T::class.java, *args)
```

Pasha Finkelshteyn @asm0di0

runApplication

```
inline fun <reified T : Any> runApplication(vararg args: String) =
    SpringApplication.run(T::class.java, *args)
```

runApplication

```
inline fun <reified T : Any> runApplication(vararg args: String) =
    SpringApplication.run(T::class.java, *args)
```

The first goodie of Spring for Kotlin

Let’s start implementing

Chapter 1. MVC + Validation

First controller

```
@RestController
@RequestMapping("/person")
class PersonController {
  @PostMapping
  fun createPerson(@RequestBody @Valid person: Person) {}
}
```

First controller

```kotlin
@RestController
@RequestMapping("/person")
class PersonController {
  @PostMapping
  fun createPerson(@RequestBody @Valid person: Person) {}
}
```

First controller

```
@RestController
@RequestMapping("/person")
class PersonController {
  @PostMapping
  fun createPerson(@RequestBody @Valid person: Person) {}
}
```

First controller

```kotlin
@RestController
@RequestMapping("/person")
class PersonController {
  @PostMapping
  fun createPerson(@RequestBody @Valid person: Person) {}
}
```

First controller

```kotlin
@RestController
@RequestMapping("/person")
class PersonController {
  @PostMapping
  fun createPerson(@RequestBody @Valid person: Person) {}
}
```

`Person.kt`:

```kotlin
data class Person(
  val name: String,
  val age: Int
)
```

Make an empty `POST`...

```
POST localhost:8080/person
Content-Type: application/json
```

Make an empty `POST`...

```
POST localhost:8080/person
Content-Type: application/json
```

Make an empty `POST`...

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "timestamp": 1674735741056,
  "status": 400,
  "error": "Bad Request",
  "path": "/person"
}
```

Make an empty `POST`...

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "timestamp": 1674735741056,
  "status": 400,
  "error": "Bad Request",
  "path": "/person"
}
```

Make an empty `POST`...

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "timestamp": 1674735741056,
  "status": 400,
  "error": "Bad Request",
  "path": "/person"
}
```

Since `Person` is non-nullable — it's validated without `@NotNull` annotation

Why? How?

`build.greadle.kts`:

```kotlin
tasks.withType<KotlinCompile> {
  kotlinOptions {
    freeCompilerArgs = listOf("-Xjsr305=strict")
    jvmTarget = "21"
  }
}
```

Why? How?

`build.greadle.kts`:

```
tasks.withType<KotlinCompile> {
  kotlinOptions {
    freeCompilerArgs = listOf("-Xjsr305=strict")
    jvmTarget = "21"
  }
}
```

Why? How?

`build.greadle.kts`:

```
tasks.withType<KotlinCompile> {
  kotlinOptions {
    freeCompilerArgs = listOf("-Xjsr305=strict")
    jvmTarget = "21"
  }
}
```

Why? How?

`build.greadle.kts`:

```
tasks.withType<KotlinCompile> {
  kotlinOptions {
    freeCompilerArgs = listOf("-Xjsr305=strict")
    jvmTarget = "21"
  }
}
```

`JSR 305: Annotations for Software Defect Detection`:

> Nullness annotations (e.g., `@NonNull` and `@CheckForNull`)

> Internationalization annotations, such as `@NonNls` or `@Nls`

Non-empty `POST` with empty properties

```
POST localhost:8080/person
Content-Type: application/json

{"name": null, "age": null}
```

Non-empty `POST` with empty properties

```
POST localhost:8080/person
Content-Type: application/json

{"name": null, "age": null}
```

On client

Non-empty `POST` with empty properties

```
POST localhost:8080/person
Content-Type: application/json

{"name": null, "age": null}
```

On client

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

Non-empty `POST` with empty properties

```
POST localhost:8080/person
Content-Type: application/json

{"name": null, "age": null}
```

On client

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

On server

Non-empty `POST` with empty properties

```
POST localhost:8080/person
Content-Type: application/json

{"name": null, "age": null}
```

On client

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

On server

```
…Instantiation of [simple type, class com.github.asm0dey.sample.Person]
  value failed for JSON property name due to missing
```

Non-empty `POST` with empty properties

```
POST localhost:8080/person
Content-Type: application/json

{"name": null, "age": null}
```

On client

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

On server

```
…Instantiation of [simple type, class com.github.asm0dey.sample.Person]
  value failed for JSON property name due to missing
```

`POST` with non-empty name

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

`POST` with non-empty name

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

`POST` with non-empty name

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

```
HTTP/1.1 200
Content-Length: 0
```

`POST` with non-empty name

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

```
HTTP/1.1 200
Content-Length: 0
```

`POST` with non-empty name

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

```
HTTP/1.1 200
Content-Length: 0
```

Rechecking

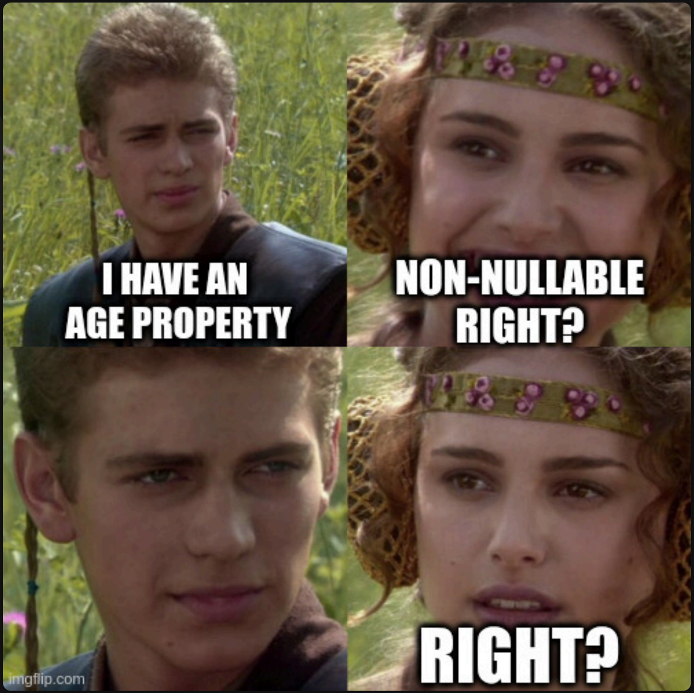
```kotlin
data class Person(
  val name: String,
  val age: Double
)
```

Rechecking

```
data class Person(
  val name: String,
  val age: Double
)
```

Rechecking

```
data class Person(
  val name: String,
  val age: Double
)
```



In JVM primitive types have default values

These types will be JVM primitives:

- `Double`
- `Int`
- `Float`
- `Char`
- `Short`
- `Byte`
- `Boolean`

Updating class

```kotlin
data class Person(
  val name: String,
  @field:NotNull val age: Double?
)
```

Updating class

```kotlin
data class Person(
  val name: String,
  @field:NotNull val age: Double?
)
```

Updating class

```kotlin
data class Person(
  val name: String,
  @field:NotNull val age: Double?
)
```

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

Updating class

```kotlin
data class Person(
  val name: String,
  @field:NotNull val age: Double?
)
```

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

Updating class

```kotlin
data class Person(
  val name: String,
  @field:NotNull val age: Double?
)
```

```http
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

```http
HTTP/1.1 400 Bad Request
…
{ "timestamp": 1674760360096, "status": 400, "error": "Bad Request", "path": "/person" }
```

Updating class

```kotlin
data class Person(
  val name: String,
  @field:NotNull val age: Double?
)
```

```
POST localhost:8080/person
Content-Type: application/json

{"name": "Pasha", "age": null}
```

```
HTTP/1.1 400 Bad Request
…
{ "timestamp": 1674760360096, "status": 400, "error": "Bad Request", "path": "/person" }
```

```
Field error in object 'person' on field 'age': rejected value [null]
```

Also…

```yaml
spring:
  jackson:
    deserialization:
      FAIL_ON_NULL_FOR_PRIMITIVES: true
```

Quick summary

- `-Xjsr305=strict` will make the validation easier
- For JVM primitive types we have to put `@field:NotNull` and mark them nullable
- Sometimes can work it around with jackson settings

JPA

Chapter 2

Nanoentity

```
@Entity
data class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

Nanoentity

```kotlin
@Entity
data class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

- `data` class

Nanoentity

```
@Entity
data class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

- `data` class
- `val name` and `val age`

Nanoentity

```kotlin
@Entity
data class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

- `data` class
- `val name` and `val age`

Improving

`data` classes have `copy`, `equals`, `hashCode`, `copy`, and `componentX` defined

```
@Entity
data class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
 )
```

Improving

`data` classes have `copy`, `equals`, `hashCode`, `copy`, and `componentX` defined

```kotlin
@Entity
data class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

Improving

```
@Entity
class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

Improving

```
@Entity
class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  val name: String,
  @Column(nullable = false)
  val age: Int,
)
```

JPA won’s be able to write to `val`

Improving

```kotlin
@Entity
class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  var name: String,
  @Column(nullable = false)
  var age: Int,
 )
```

JPA won’s be able to write to `val`

Improving

```
@Entity
class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  var name: String,
  @Column(nullable = false)
  var age: Int,
)
```

JPA won’s be able to write to `val`

But there is no no-arg constructor!

How to make it work?

Magic:

```
kotlin("plugin.jpa") version "1.8.0"
```

But there is no no-arg constructor!

How to make it work?

Magic:

```
kotlin("plugin.jpa") version "1.8.0"
```

- Puts annotations on the fields
- Adds a default constructor in bytecode*!

But there is no no-arg constructor!

How to make it work?

Magic:

```
kotlin("plugin.jpa") version "1.8.0"
```

- Puts annotations on the fields
- Adds a default constructor in bytecode*!

* In Kotlin the default constructor would not be possible, but in Java it is

Current result

```kotlin
@Entity
class Person(
  @Id
  @GeneratedValue(strategy = IDENTITY)
  var id: Int? = null,
  @Column(nullable = false)
  var name: String,
  @Column(nullable = false)
  var age: Int,
)
```

Is this enough?

Not quite.

At the very least we have to redefine `equals` and `hashCode`.

For example…

```kotlin
@Entity
class Person(
  // properties
) {
  // equals…
  override fun hashCode(): Int {
    return id ?: 0
  }
}
```

Is this enough?

Not quite.

At the very least we have to redefine `equals` and `hashCode`.

For example…

```
@Entity
class Person(
  // properties
) {
  // equals…
  override fun hashCode(): Int {
    return id ?: 0
  }
}
```

Is this enough?

Not quite.

At the very least we have to redefine `equals` and `hashCode`.

For example…

```
@Entity
class Person(
  // properties
) {
  // equals…
  override fun hashCode(): Int {
    return id ?: 0
  }
}
```

Is this enough?

Not quite.

At the very least we have to redefine `equals` and `hashCode`.

For example…

```
@Entity
class Person(
  // properties
) {
  // equals…
  override fun hashCode(): Int {
    return id ?: 0
  }
}
```

JDBC

Chapter 3

Obtain user by id

Let's imagine we need to call the following:

```sql
SELECT *
FROM   users
WHERE id = ?
```

Or, for example…

Or, for example…

In Java

Let’s inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let’s inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let's inline mapper

```java
public List<Person> findById(int id) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", new UserRowMapper(), id);
}

private static class UserRowMapper implements RowMapper<Person> {
  @Override
  public Person mapRow(ResultSet resultSet, int i) throws SQLException {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id, name, age);
  }
}
```

In Java

☕ Let's inline mapper

```java
public List<Person> findById(int userId) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", (resultSet, i) -> {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id1, name, age);
  }, userId);
}
```

In Java

```java
public List<Person> findById(int userId) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", (resultSet, i) -> {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id1, name, age);
  }, userId);
}
```

In Java

```java
public List<Person> findById(int userId) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", (resultSet, i) -> {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id1, name, age);
  }, userId);
}
```

In Java

```java
public List<Person> findById(int userId) {
  return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", (resultSet, i) -> {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    Double age = resultSet.getDouble("age");
    return new Person(id1, name, age);
  }, userId);
}
```

I don’t like it

- Too many mappers
- Parameters are too far from query

😭

Why should it be so?

Why should it be so?

Let’s Look at the signature

Why should it be so?

Let’s Look at the signature

```java
public <T> List<T> query(String sql, RowMapper<T> rowMapper, @Nullable Object... args)
```

Why should it be so?

Let’s Look at the signature

```
public <T> List<T> query(String sql, RowMapper<T> rowMapper, @Nullable Object... args)
```

Because in ☕ vararg can be only the last… 😥

`JdbcTemplate` in Kotlin

```kotlin
return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", userId) { rs, _ ->
  val id = rs.getInt("id")
  val name = rs.getString("name")
  val age = rs.getDouble("age")
  Person(id, name, age)
}
```

`JdbcTemplate` in Kotlin

```kotlin
return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", userId) { rs, _ ->
  val id = rs.getInt("id")
  val name = rs.getString("name")
  val age = rs.getDouble("age")
  Person(id, name, age)
}
```

`JdbcTemplate` in Kotlin

```kotlin
return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", userId) { rs, _ ->
  val id = rs.getInt("id")
  val name = rs.getString("name")
  val age = rs.getDouble("age")
  Person(id, name, age)
}
```

`JdbcTemplate` in Kotlin

```
return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", userId) { rs, _ ->
    val id = rs.getInt("id")
    val name = rs.getString("name")
    val age = rs.getDouble("age")
    Person(id, name, age)
}
```

- `vararg` doesn't have to be in the last position
- unused parameter of a lambda can be named `_`

Extension functions

```kotlin
fun <T> JdbcOperations.query(
  sql: String,
  vararg args: Any,
  function: (ResultSet, Int) -> T
): List<T>
```

Extension functions

```kotlin
fun <T> JdbcOperations.query(
  sql: String,
  vararg args: Any,
  function: (ResultSet, Int) -> T
): List<T>
```

Extension functions

```
fun <T> JdbcOperations.query(
  sql: String,
  vararg args: Any,
  function: (ResultSet, Int) -> T
): List<T>
```

Extension functions

```
fun <T> JdbcOperations.query(
  sql: String,
  vararg args: Any,
  function: (ResultSet, Int) -> T
): List<T>
```

Extension functions

```
fun <T> JdbcOperations.query(
  sql: String,
  vararg args: Any,
  function: (ResultSet, Int) -> T
): List<T>
```

Pasha Finkelshteyn @asm0di0

Extension functions

```kotlin
fun <T> JdbcOperations.query(
  sql: String,
  vararg args: Any,
  function: (ResultSet, Int) -> T
): List<T>
```

Which allows

```kotlin
return jdbcTemplate.query("SELECT * FROM users WHERE id = ?", userId)
{ rs, _ ->
  // TODO: ResultSet → Person
}
```

You can do the same for your own code!

From ☕

```
public List<String> transformStrings(Function<String, String> mapper, String... args){}
```

You can do the same for your own code!

From Java

```java
public List<String> transformStrings(Function<String, String> mapper, String... args){}
```

To Kotlin

```kotlin
fun transformStrings(vararg args: String, mapper: (String) -> String): List<String>{}
```

More on extensions for Spring

- spring-beans
- spring-context
- spring-core
- spring-jdbc
- spring-messaging
- spring-r2dbc
- spring-test
- spring-tx
- spring-web
- spring-webflux
- spring-webmvc

Reactive persistence

Chapter 4

Who knows what is reactive?

Who knows what is reactive?

- non-blocking

Who knows what is reactive?

- non-blocking
- asynchronous

Who knows what is reactive?

- non-blocking
- asynchronous
- handles back-pressure

Who knows what is reactive?

- non-blocking
- asynchronous
- handles back-pressure

The result is a monad…

Who knows what is reactive?

- non-blocking
- asynchronous
- handles back-pressure

The result is either:

- `Mono` : produces 0 to 1 items when it’s ready
- `Flux` : Produces 0 to ∞ items

In Kotlin

Coroutines operate on `suspend` functions

`suspend` can be called from:

- coroutine context
- another `suspend` `fun`

returns usual types - Lists, Strings, etc

```kotlin
suspend fun randomNum(): Int {
  val x = service1.randomNum() // suspend
  val y = service2.randomNum() // suspend
  return x + y
}
```

In Kotlin

Coroutines operate on `suspend` functions

`suspend` can be called from:

- coroutine context
- another `suspend` `fun`

returns usual types - Lists, Strings, etc

```
suspend fun randomNum(): Int {
  val x = service1.randomNum() // suspend
  val y = service2.randomNum() // suspend
  return x + y
}
```

In Kotlin

Coroutines operate on `suspend` functions

`suspend` can be called from:

- coroutine context
- another `suspend` `fun`

returns usual types - Lists, Strings, etc

```kotlin
suspend fun randomNum(): Int {
  val x = service1.randomNum() // suspend
  val y = service2.randomNum() // suspend
  return x + y
}
```

In Kotlin

Coroutines operate on `suspend` functions

`suspend` can be called from:

- coroutine context
- another `suspend` `fun`

returns usual types - Lists, Strings, etc

```kotlin
suspend fun randomNum(): Int {
  val x = service1.randomNum() // suspend
  val y = service2.randomNum() // suspend
  return x + y
}
```


SUSPEND
SUSPEND EVERYWHERE
imgflip.com

Spring R2DBC

```
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
}
```

Spring R2DBC

```
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id"
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id"
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
 }
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id")
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id")
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
}
```

Spring R2DBC

```
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id"
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
    // inside Spring ↓
    suspend fun <T> RowsFetchSpec<T>.awaitSingle(): T {
      return first().awaitSingleOrNull() ?: throw EmptyResultDataAccessException(1)
    }
}
```

Spring R2DBC

```kotlin
@Repository
class Repo(connectionFactory: ConnectionFactory) {
    val client = DatabaseClient.create(connectionFactory)
    suspend fun createUserAndReturnId() =
        client
            .sql("INSERT INTO users (name, email, age) VALUES ('Pasha', :email, NULL) RETURNING id")
            .bind("email", RandomStringUtils.randomAlphabetic(20))
            .fetch()
            .awaitSingle()["id"] as? Long ?: error("not long on not returned")
}
```

Transactions

```kotlin
@Transactional
suspend fun failTransactional(): Long {
    val curId =  repo.createUserAndReturnId()
    repo.createUserWithConflictingId(curId) // will throw
    return curId
}
```

Transactions

```
@Transactional
suspend fun failTransactional(): Long {
    val curId =  repo.createUserAndReturnId()
    repo.createUserWithConflictingId(curId) // will throw
    return curId
}
```

Transactions

```kotlin
@Transactional
suspend fun failTransactional(): Long {
    val curId =  repo.createUserAndReturnId()
    repo.createUserWithConflictingId(curId) // will throw
    return curId
}
```

Transactions

```kotlin
@Transactional
suspend fun failTransactional(): Long {
    val curId =  repo.createUserAndReturnId()
    repo.createUserWithConflictingId(curId) // will throw
    return curId
}
```

Transactions

```kotlin
@Transactional
suspend fun failTransactional(): Long {
    val curId =  repo.createUserAndReturnId()
    repo.createUserWithConflictingId(curId) // will throw
    return curId
}
```

Nothing changes from the client standpoint!

Repositories

```kotlin
interface User : CoroutineCrudRepository<User, Long> {
}
```

Repositories

```kotlin
interface User : CoroutineCrudRepository<User, Long> {
}
// Inside Spring
interface CoroutineCrudRepository<T, ID> : Repository<T, ID> {
  // ...
  suspend fun <S : T> save(entity: S): T
  // Flow is an async unbounded collection
  fun findAll(): Flow<T>
  // ...
}
```

Repositories

```kotlin
interface User : CoroutineCrudRepository<User, Long> {
}
// Inside Spring
interface CoroutineCrudRepository<T, ID> : Repository<T, ID> {
  // ...
  suspend fun <S : T> save(entity: S): T
  // Flow is an async unbounded collection
  fun findAll(): Flow<T>
  // ...
}
```

Repositories

```
interface User : CoroutineCrudRepository<User, Long> {
}
// Inside Spring
interface CoroutineCrudRepository<T, ID> : Repository<T, ID> {
  // ...
  suspend fun <S : T> save(entity: S): T
  // Flow is an async unbounded collection
  fun findAll(): Flow<T>
  // ...
 }
```

Repositories

```
interface User : CoroutineCrudRepository<User, Long> {
}
```

Repositories

```kotlin
interface User : CoroutineCrudRepository<User, Long> {
  suspend fun findOne(id: String): User
  fun findByFirstname(firstname: String): Flow<User>
  suspend fun findAllByFirstname(id: String): List<User>
}
```

Repositories

```kotlin
interface User : CoroutineCrudRepository<User, Long> {
  suspend fun findOne(id: String): User
  fun findByFirstname(firstname: String): Flow<User>
  suspend fun findAllByFirstname(id: String): List<User>
}
```

Repositories

```
interface User : CoroutineCrudRepository<User, Long> {
  suspend fun findOne(id: String): User
  fun findByFirstname(firstname: String): Flow<User>
  suspend fun findAllByFirstname(id: String): List<User>
}
```

Repositories

```
interface User : CoroutineCrudRepository<User, Long> {
  suspend fun findOne(id: String): User
  fun findByFirstname(firstname: String): Flow<User>
  suspend fun findAllByFirstname(id: String): List<User>
}
```

Function is either suspend or returns `Flow`

Transactions with repositories?

```
@Transactional
suspend fun failTransactional(): Long {
  val u = User(null, "me@asm0dey.site", 37)
  val savedId =  repo.save(u)
  repo.save(u.copy(id = savedId)) // will throw
  return curId
}
```

Transactions with repositories?

```kotlin
@Transactional
suspend fun failTransactional(): Long {
  val u = User(null, "me@asm0dey.site", 37)
  val savedId =  repo.save(u)
  repo.save(u.copy(id = savedId)) // will throw
  return curId
}
```

Transactions with repositories?

```
@Transactional
suspend fun failTransactional(): Long {
  val u = User(null, "me@asm0dey.site", 37)
  val savedId =  repo.save(u)
  repo.save(u.copy(id = savedId)) // will throw
  return curId
}
```

Transactions with repositories?

```
@Transactional
suspend fun failTransactional(): Long {
  val u = User(null, "me@asm0dey.site", 37)
  val savedId =  repo.save(u)
  repo.save(u.copy(id = savedId)) // will throw
  return curId
}
```

Transactions with repositories?

```
@Transactional
suspend fun failTransactional(): Long {
  val u = User(null, "me@asm0dey.site", 37)
  val savedId =  repo.save(u)
  repo.save(u.copy(id = savedId)) // will throw
  return curId
}
```

Configuration

Let’s start simple

```
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Let's start simple

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Let's start simple

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Modified Jackson's `ObjectMapper` to work with `data` classes from `jackson-module-kotlin`

```kotlin
@Bean
fun kotlinMapper(): ObjectMapper {
  return jacksonObjectMapper()
}
```

Let's start simple

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Modified Jackson's `ObjectMapper` to work with `data` classes from `jackson-module-kotlin`

```kotlin
@Bean
fun kotlinMapper(): ObjectMapper {
  return jacksonObjectMapper()
}
```

Let’s start simple

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Modified Jackson’s `ObjectMapper` to work with `data` classes from `jackson-module-kotlin`

```kotlin
@Bean
fun kotlinMapper(): ObjectMapper {
  return jacksonObjectMapper()
}
```

Let’s start simple

```
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Modified Jackson’s `ObjectMapper` to work with `data` classes from `jackson-module-kotlin`

```
@Bean
fun kotlinMapper(): ObjectMapper {
  return jacksonObjectMapper()
}
```

Let's start simple

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
}
```

Modified Jackson's `ObjectMapper` to work with `data` classes from `jackson-module-kotlin`

```kotlin
@Bean
fun kotlinMapper(): ObjectMapper {
  return jacksonObjectMapper()
}
```

4 lines instead of 1 😱

Custom bean

```kotlin
class JsonLogger(private val objectMapper: ObjectMapper) {
  fun log(o: Any) {
    if (o::class.isData) {
      println(objectMapper.writeValueAsString(o))
    } else println(o.toString())
  }
}

val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
}
```

Custom bean

```kotlin
class JsonLogger(private val objectMapper: ObjectMapper) {
  fun log(o: Any) {
    if (o::class.isData) {
      println(objectMapper.writeValueAsString(o))
    } else println(o.toString())
  }
}

val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
}
```

Custom bean

```kotlin
class JsonLogger(private val objectMapper: ObjectMapper) {
  fun log(o: Any) {
    if (o::class.isData) {
      println(objectMapper.writeValueAsString(o))
    } else println(o.toString())
  }
}

val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
}
```

Custom bean

```kotlin
class JsonLogger(private val objectMapper: ObjectMapper) {
  fun log(o: Any) {
    if (o::class.isData) {
      println(objectMapper.writeValueAsString(o))
    } else println(o.toString())
  }
}

val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
}
```

Custom bean

```
class JsonLogger(private val objectMapper: ObjectMapper) {
  fun log(o: Any) {
    if (o::class.isData) {
      println(objectMapper.writeValueAsString(o))
    } else println(o.toString())
  }
}

val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
}
```

Arbitrary logic

```
val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
  bean("randomGoodThing", isLazyInit = Random.nextBoolean()) {
    if (Random.nextBoolean()) "Norway" else "Well"
  }
}
```

Arbitrary logic

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
  bean("randomGoodThing", isLazyInit = Random.nextBoolean()) {
    if (Random.nextBoolean()) "Norway" else "Well"
  }
}
```

Arbitrary logic

```kotlin
val beans = beans {
  bean { jacksonObjectMapper() }
  bean(::JsonLogger)
  bean("randomGoodThing", isLazyInit = Random.nextBoolean()) {
    if (Random.nextBoolean()) "Norway" else "Well"
  }
}
```

OK How do I use it?

Let’s return to our very first file

```
runApplication<SampleApplication>(*args)
```

```
val beans = { /* */ }
```

OK How do I use it?

Let’s return to our very first file

```
runApplication<SampleApplication>(*args)
```

```
val beans = { /* */ }
```

Let’s change it to

```
runApplication<SampleApplication>(*args) {
  addInitializers(beans)
}
```

OK How do I use it?

Let's return to our very first file

```kotlin
runApplication<SampleApplication>(*args)
```

```kotlin
val beans = { /* */ }
```

Let's change it to

```kotlin
runApplication<SampleApplication>(*args) {
  addInitializers(beans)
}
```

OK How do I use it?

Let’s return to our very first file

```
runApplication<SampleApplication>(*args)
```

```
val beans = { /* */ }
```

Let’s change it to

```
runApplication<SampleApplication>(*args) {
  addInitializers(beans)
}
```

And run it…

```
Started SampleApplicationKt in 1.776 seconds (process running for 2.133)
```

Let's test it

Bean:

```kotlin
@Component
class MyBean(val jsonLogger: JsonLogger) {
  fun test() = jsonLogger.log("Test")
}
```

Test:

```kotlin
@SpringBootTest
class ConfigTest {
  @Autowired private lateinit var myBean: MyBean
  @Test
  fun testIt() = assertEquals("Test", myBean.test())
}
```

Let's test it

Bean:

```
@Component
class MyBean(val jsonLogger: JsonLogger) {
  fun test() = jsonLogger.log("Test")
}
```

Test:

```
@SpringBootTest
class ConfigTest {
  @Autowired private lateinit var myBean: MyBean
  @Test
  fun testIt() = assertEquals("Test", myBean.test())
}
```

Let's test it

Bean:

```kotlin
@Component
class MyBean(val jsonLogger: JsonLogger) {
  fun test() = jsonLogger.log("Test")
}
```

Test:

```kotlin
@SpringBootTest
class ConfigTest {
  @Autowired private lateinit var myBean: MyBean
  @Test
  fun testIt() = assertEquals("Test", myBean.test())
}
```

Let’s test it

Bean:

```kotlin
@Component
class MyBean(val jsonLogger: JsonLogger) {
  fun test() = jsonLogger.log("Test")
}
```

Test:

```kotlin
@SpringBootTest
class ConfigTest {
  @Autowired private lateinit var myBean: MyBean
  @Test
  fun testIt() = assertEquals("Test", myBean.test())
}
```

Let’s test it

Bean:

```
@Component
class MyBean(val jsonLogger: JsonLogger) {
  fun test() = jsonLogger.log("Test")
}
```

Test:

```
@SpringBootTest
class ConfigTest {
  @Autowired private lateinit var myBean: MyBean
  @Test
  fun testIt() = assertEquals("Test", myBean.test())
}
```

Let’s test it

Bean:

```kotlin
@Component
class MyBean(val jsonLogger: JsonLogger) {
  fun test() = jsonLogger.log("Test")
}
```

Test:

```kotlin
@SpringBootTest
class ConfigTest {
  @Autowired private lateinit var myBean: MyBean
  @Test
  fun testIt() = assertEquals("Test", myBean.test())
}
```

Run it

```
No qualifying bean of type
'com.github.asm0dey.sample.JsonLogger'
  available: expected at least 1
  bean which qualifies as autowire candidate
```

Run it

```
No qualifying bean of type
'com.github.asm0dey.sample.JsonLogger'
  available: expected at least 1
  bean which qualifies as autowire candidate
```

Run it

```
No qualifying bean of type
'com.github.asm0dey.sample.JsonLogger'
  available: expected at least 1
  bean which qualifies as autowire candidate
```

That’s because our tests do not call `main`!

Requires some glue to work

```kotlin
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

Requires some glue to work

```kotlin
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

Requires some glue to work

```
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

Requires some glue to work

```kotlin
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

Requires some glue to work

```kotlin
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

`application.yml`:

```yaml
context.initializer.classes: "com.github.asm0dey.sample.BeansInitializer"
```

Requires some glue to work

```kotlin
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

`application.yml`:

```yaml
context.initializer.classes: "com.github.asm0dey.sample.BeansInitializer"
```

`Main.kt`:

```kotlin
fun main(args: Array<String>) {
  runApplication<SampleApplication>(*args)
}
```

Requires some glue to work

```kotlin
val beans = { /* */ }
class BeansInitializer : ApplicationContextInitializer<GenericApplicationContext> {
  override fun initialize(context: GenericApplicationContext) = beans.initialize(context)
}
```

`application.yml`:

```yaml
context.initializer.classes: "com.github.asm0dey.sample.BeansInitializer"
```

`Main.kt`:

```kotlin
fun main(args: Array<String>) {
  runApplication<SampleApplication>(*args)
}
```

Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

Spring Security

So, what did I learn?

So, what did I learn?

So, what did I learn?

- Always generate the project with start.spring.io

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better
- Validation is better with Kotlin, but remember about primitives

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better
- Validation is better with Kotlin, but remember about primitives
- `data` classes should not be used for JPA

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better
- Validation is better with Kotlin, but remember about primitives
- `data` classes should not be used for JPA
- JDBC is simpler with Kotlin

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better
- Validation is better with Kotlin, but remember about primitives
- `data` classes should not be used for JPA
- JDBC is simpler with Kotlin
- R2DBC is simpler with coroutines

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better
- Validation is better with Kotlin, but remember about primitives
- `data` classes should not be used for JPA
- JDBC is simpler with Kotlin
- R2DBC is simpler with coroutines
- Bean definition DSL is awesome

So, what did I learn?

- Always generate the project with start.spring.io
- Reified generics might make an API better
- Validation is better with Kotlin, but remember about primitives
- `data` classes should not be used for JPA
- JDBC is simpler with Kotlin
- R2DBC is simpler with coroutines
- Bean definition DSL is awesome
- Specifically with security!

Thank you!

Thank you! Questions?

- asm0di0
- @asm0dey@fosstodon.org
- me@asm0dey.site
- asm0dey
- asm0dey
- asm0dey
- asm0dey
- asm0dey

END